ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

UNIT 2: Xây dựng giao diện ứng dụng

ONE LOVE. ONE FUTURE.

Mục tiêu

Nền tảng Kotlin

- Điều kiện và null safety
- Lớp, đối tượng, kế thừa
- Lambda và hàm bậc cao

Ứng dụng Compose

- Button, Image, TextField, Switch
- State và recomposition
- Debugger và automated tests

HUST

hust.edu.vn fb.com/dhbkhn

PATHWAY 1

Kotlin fundamentals

Lập trình hướng đối tượng

Hàm lambda

PATHWAY 2

Add a button to an app

- Dice Roller
- Debugger
- Click behavior

Luồng tương tác Dice Roller

Đầu vào

Người dùng nhấn Roll

Xử lý

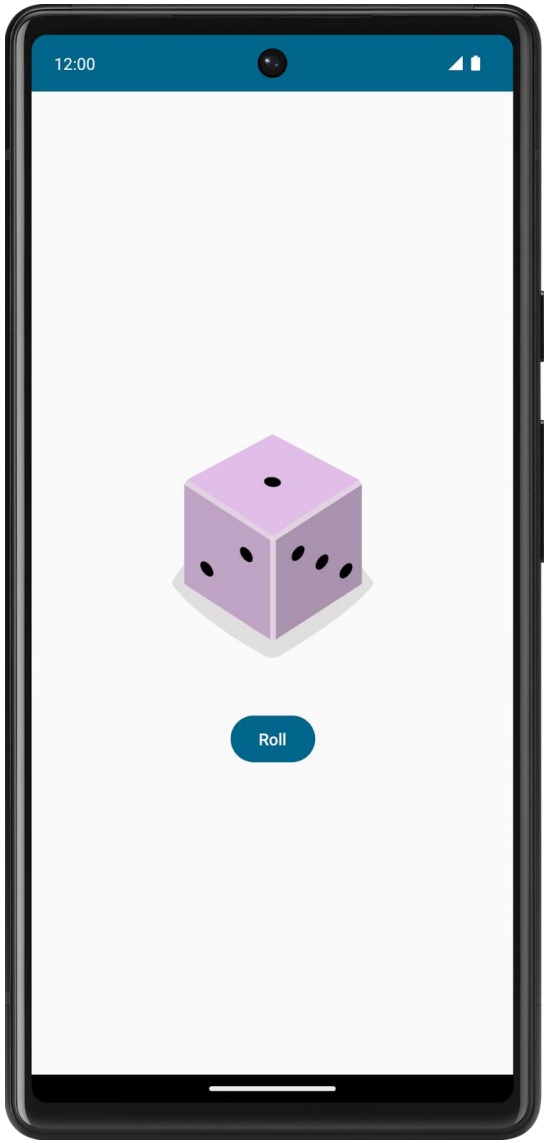
Sinh số ngẫu nhiên 1..6

Đầu ra

Chọn drawable tương ứng

UI

Image cập nhật sau recomposition



Thiết lập project Dice Roller

https://developer.android.com/codelabs/basic-android-kotlin-compose-build-a-dice-roller-app

Thêm button vào ứng dụng

```
Column(
    modifier = modifier,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Button(onClick = { /*TODO*/ }) {
        Text(stringResource(R.string.roll))
    }
}
```

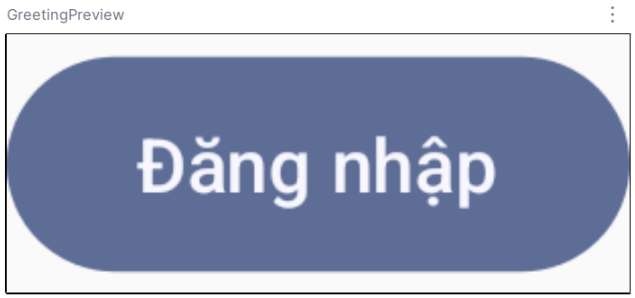
Hàm Button()

Button() là gì?

Button là composable dùng để tạo nút bấm và xử lý hành động người dùng.

Cú pháp cơ bản

```
Button(onClick = { /* xử lý sự kiện */ }) {
    Text("Đăng nhập")
}
```



Các thành phần chính

- **onClick:** hàm được gọi khi người dùng nhấn nút.
- **enabled:** bật/tắt khả năng tương tác của nút.
- **colors, shape, border:** tùy biến màu sắc, hình dạng và đường viền.
- **content:** nội dung bên trong nút, thường là Text hoặc Icon + Text.

Thay đổi thiết kế của Button

```
Button(
    onClick = {  },
    border = BorderStroke(2.dp, Color.Red),
    shape = RoundedCornerShape(CornerSize(2.dp)),
    colors = ButtonDefaults.buttonColors(
        containerColor = Color.Blue,
        contentColor = Color.White,
    )
) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Icon(painter = painterResource(R.drawable.android_48dp), contentDescription = null)
        Spacer(Modifier.width(8.dp))
        Text("Đăng nhập", fontSize = 18.sp)
    }
}
```



Khoảng cách với Spacer

```
Image(...)
Spacer(modifier = Modifier.height(16.dp))
Button(...)
```

- **dp** dùng cho kích thước bố cục
- Thường tăng theo bội số **4.dp**
- Tránh dùng khoảng trắng trong chuỗi để căn UI

Vì sao biến cục bộ chưa đủ?

Không nhớ trạng thái

```
var result = 1
```

Recomposition có thể tạo lại biến và đưa giá trị về 1.

Có nhớ trạng thái

```
var result by remember {
    mutableStateOf(1)
}
```

Giá trị biến được giữ lại sau khi recomposition.

Thay đổi result kích hoạt cập nhật UI.

Recomposition

Dòng xử lý: State thay đổi → scope bị invalidate → Compose lên lịch → chạy lại Composable liên quan → cập nhật UI

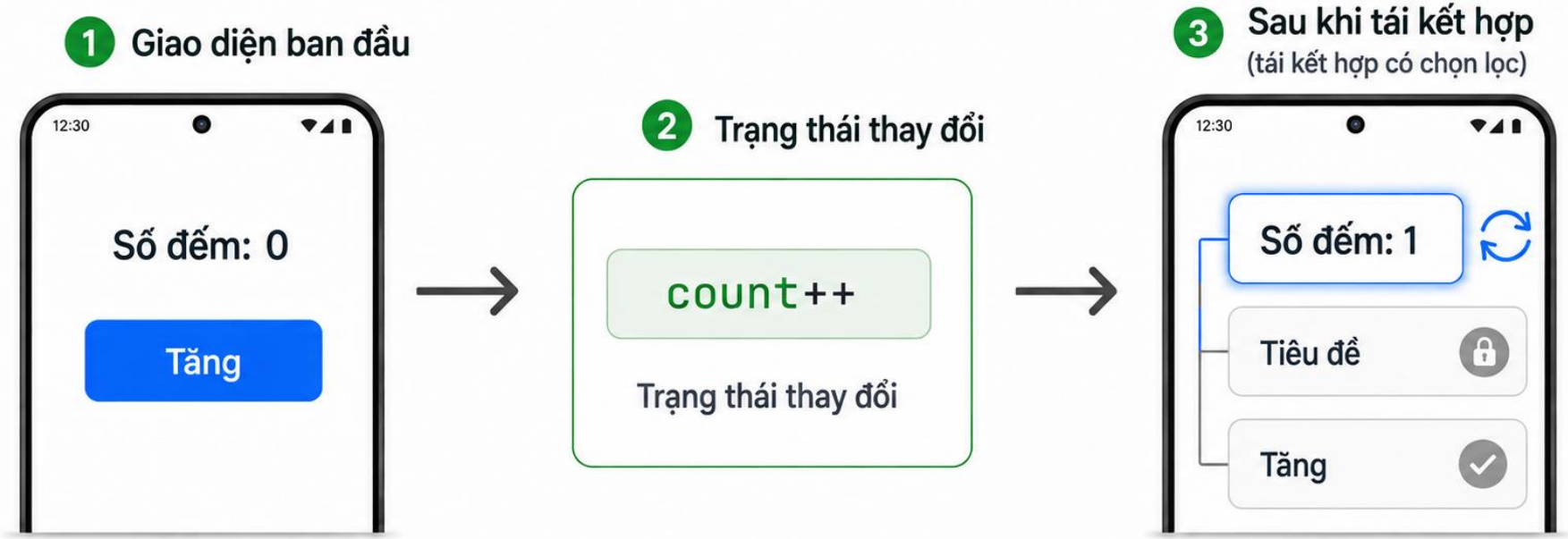
- **Ghi nhận phụ thuộc:** trong Composition, Compose theo dõi Composable nào đã đọc từng State.
- **Cập nhật:** khi State đổi, chỉ các recomposition scope đã đọc State đó bị đánh dấu chạy lại.
- **Bỏ qua phần ổn định:** các nhánh không phụ thuộc hoặc có input không đổi có thể được skip.
- **Áp dụng thay đổi:** Compose cập nhật cây UI; layout/draw chỉ chạy lại khi kết quả tương ứng thực sự thay đổi.

remember và recomposition

```
var result by remember {
    mutableStateOf(1)
}
```

- **remember** lưu object qua các lần recomposition
- **mutableStateOf** tạo state quan sát được
- **by** cho phép đọc/ghi trực tiếp như một biến

remember và recomposition




```
1  var count by remember { mutableIntStateOf(0) }
2  Text("Số đếm: $count")
3  Button(onClick = { count++ })
```

Chỉ những composable đọc trạng thái đã thay đổi mới được tái kết hợp

Hàm clickable() của modifier

Cho phép bắt sự kiện nhấn trên bất kỳ đối tượng nào như Text, Image, …

Cú pháp:

```
public fun Modifier.clickable(
    enabled: Boolean = true,
    onClickLabel: String? = null,
    role: Role? = null,
    interactionSource: MutableInteractionSource? = null,
    onClick: () -> Unit
): Modifier
```

Ví dụ

```
Text(
    text = "Click here",
    modifier = Modifier.clickable {
        // Xử lý sự kiện
    }
)
```



Debugger quan sát runtime

Dùng debugger khi cần

- Xem state thực tế tại một thời điểm
- Xác định nhánh nào được thực thi
- Theo dõi giá trị thay đổi qua từng dòng
- Hiểu logic error dù ứng dụng vẫn build thành công

Kiểm tra biến khi debug

Ví dụ khi Dice Roller dừng

- **result** đang bằng bao nhiêu?
- **imageResource** trỏ tới **drawable** nào?
- Nhánh when nào vừa được chọn?

Evaluate Expression cho phép thử biểu thức mà không sửa source.

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-build-a-dice-roller-app

https://developer.android.com/codelabs/basic-android-kotlin-compose-button-click-practice-problem

PATHWAY 3

Interact with UI and state

- Form controls
  - TextField
  - CheckBox
  - Switch
  - RadioButton
- State hoisting

State quyết định UI

Ví dụ state

- Nội dung TextField
- Switch bật/tắt
- Kết quả tip
- Màn hình hiện tại

Khi state thay đổi

Compose lên lịch recomposition

→ đọc state mới

→ cập nhật phần UI phụ thuộc

Các đối tượng UI nhận tương tác từ người dùng

Button - Sự kiện nhấn nút

TextField - Nhập dữ liệu văn bản

CheckBox, **Switch** - Lựa chọn kiểu true/false

RadioButton - Lựa chọn 1 trong nhiều

Slider - Lựa chọn số trong 1 dải giá trị

TextField

```
var username by remember { mutableStateOf("admin") }
TextField(
    value = username,
    onValueChange = {username = it},
    label = { Text("Username") },
    modifier = Modifier.fillMaxWidth()
)
```

value giá trị hiện tại của TextField

onValueChange hàm callback, được gọi khi giá trị TextField thay đổi



KeyboardOptions cho TextField

KeyboardOptions cấu hình cách bàn phím ảo (IME) hiển thị và nút hành động phù hợp với dữ liệu cần nhập.

VÍ DỤ: Ô NHẬP EMAIL

```
var email by remember { mutableStateOf("") }

TextField(
    value = email,
    onValueChange = { email = it },
    label = { Text("Email") },
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Email,
        imeAction = ImeAction.Done,
        autoCorrectEnabled = false
    )
)
```

THAM SỐ THƯỜNG DÙNG

keyboardType

Chọn bố cục bàn phím: Text, Number, Phone, Email, Password...

imeAction

Đặt nút hành động: Next, Done, Search, Send...

capitalization

Tự viết hoa ký tự, từ hoặc câu khi nhập văn bản.

autoCorrectEnabled

Bật/tắt gợi ý sửa lỗi chính tả khi nhập.

VisualTransformation cho TextField

VisualTransformation thay đổi cách chuỗi được hiển thị nhưng không làm thay đổi giá trị thật đang lưu trong state.

VÍ DỤ: CHE MẬT KHẨU

```
var password by remember { mutableStateOf("") }

TextField(
    value = password,
    onValueChange = { password = it },
    label = { Text("Mật khẩu") },
    singleLine = true,
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Password
    ),
    visualTransformation =
        PasswordVisualTransformation()
)
```

CÁCH HOẠT ĐỘNG

Giá trị thật

`abc123`

Hiển thị trên giao diện

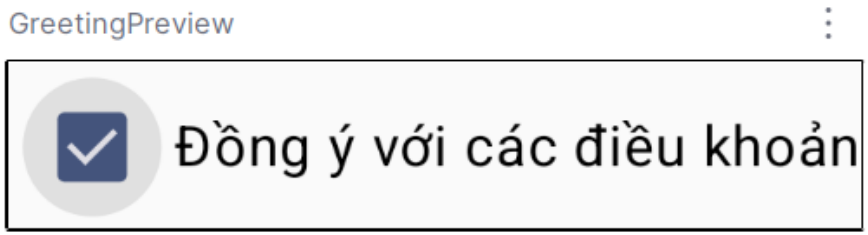
●●●●●●

PasswordVisualTransformation thay mỗi ký tự bằng dấu che. onValueChange vẫn nhận chuỗi gốc “abc123”.

Checkbox

Checkbox biểu diễn một lựa chọn đúng/sai. Trạng thái được điều khiển bởi **checked** và cập nhật qua **onCheckedChange**.

```
var accepted by remember { mutableStateOf(true) }
Row (verticalAlignment = Alignment.CenterVertically) {
    Checkbox(
        checked = accepted,
        onCheckedChange = {accepted = it},
    )
    Text("Đồng ý với các điều khoản")
}
```

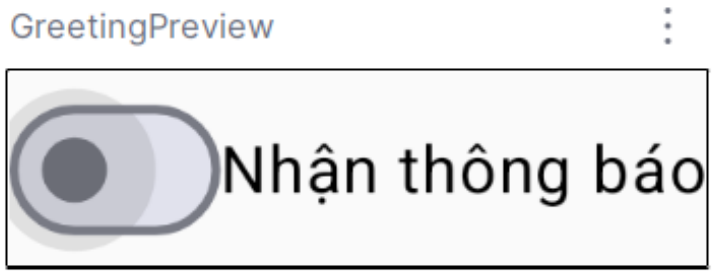


Lưu ý Checkbox không tự hiển thị nhãn. Thường đặt Checkbox và Text trong Row; có thể dùng enabled = false để vô hiệu hóa lựa chọn.

Switch

Switch cho phép bật/tắt một trạng thái. Giá trị được điều khiển bởi **checked** và cập nhật qua **onCheckedChange**.

```
var notifications by remember { mutableStateOf(false) }
Row (verticalAlignment = Alignment.CenterVertically) {
    Switch(
        checked = notifications,
        onCheckedChange = { notifications = it },
    )
    Text("Nhận thông báo")
}
```

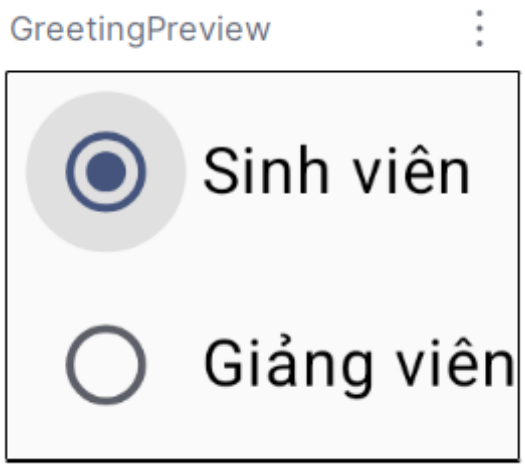


Lưu ý Switch phù hợp với cài đặt có hiệu lực ngay. Nên đặt nhãn mô tả bên cạnh; có thể dùng enabled = false để vô hiệu hóa.

RadioButton

RadioButton dùng để chọn một phương án trong nhóm. Trạng thái được điều khiển bởi **selected** và **onClick**.

```
var selected by remember { mutableStateOf("Sinh viên") }
Column {
    listOf("Sinh viên", "Giảng viên").forEach { option ->
        Row(verticalAlignment = Alignment.CenterVertically) {
            RadioButton(
                selected = selected == option,
                onClick = { selected = option }
            )
            Text(option)
        }
    }
}
```

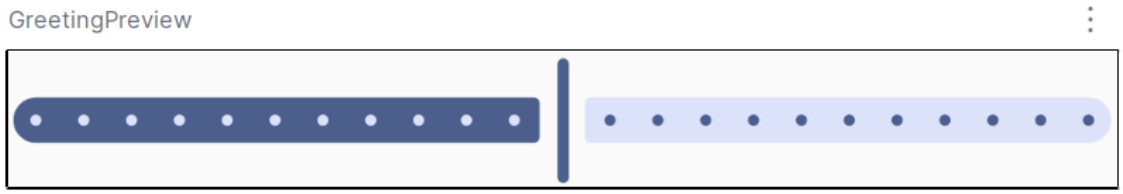


Lưu ý RadioButton thường dùng theo nhóm và chỉ nên có một lựa chọn được chọn. Đặt nhãn cạnh nút để mô tả rõ từng phương án.

Slider

Slider cho phép chọn một giá trị trong khoảng liên tục. Giá trị được điều khiển bởi **value** và **onValueChange**.

```
var volume by remember { mutableFloatStateOf(50f) }
Column {
    Slider(
        value = volume,
        onValueChange = {volume = it},
        valueRange = 0f..100f,
        steps = 21
    )
}
```



Lưu ý Dùng **valueRange** để giới hạn khoảng giá trị và **steps** để chia thành các mức. Nên hiển thị giá trị hiện tại cạnh Slider.

Vấn đề khi quản lý trạng thái

Xem xét ví dụ sau:

```
@Composable
fun Counter() {
    var count by remember { mutableIntStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}
```

Nhận xét:

- **Counter** vừa hiển thị giao diện, vừa quản lý dữ liệu.
- Composable cha không đọc hoặc thay đổi được count.
- Khó đồng bộ **count** với thành phần khác.
- Giảm khả năng tái sử dụng của composable.

State hoisting là gì?

State Hoisting là kỹ thuật chuyển state từ composable con lên composable cha.

Composable con thường nhận hai tham số:

```
@Composable
fun Counter(
    count: Int,
    onIncrement: () -> Unit
)
```

Trong đó:

- **count**: trạng thái hiện tại được truyền từ cha xuống.
- **onIncrement**: sự kiện được gửi từ con lên cha.
- Composable con trở thành **stateless composable**.
- Composable cha là nơi sở hữu và cập nhật state.

Vai trò của kỹ thuật State Hoisting

State Hoisting tách quản lý dữ liệu khỏi việc hiển thị giao diện.

- **Single source of truth:** chỉ có một nguồn dữ liệu chính.
- **Tái sử dụng:** cùng composable có thể nhận nhiều state khác nhau.
- **Dễ kiểm thử:** truyền trực tiếp dữ liệu cần kiểm tra.
- **Dễ đồng bộ:** nhiều composable có thể sử dụng chung state.
- **Dễ bảo trì:** tách UI khỏi logic quản lý trạng thái.
- **Unidirectional Data Flow:** luồng dữ liệu có một chiều rõ ràng.

Không nhất thiết phải hoisting khi:

- State chỉ là chi tiết nội bộ của một thành phần.
- State không ảnh hưởng đến phần còn lại của màn hình.
- Composable cha không cần đọc hoặc thay đổi state đó.

Stateful và stateless

Stateful

- Sở hữu remember state
- Điều phối nhiều thành phần
- Tính ***derived values***

Stateless

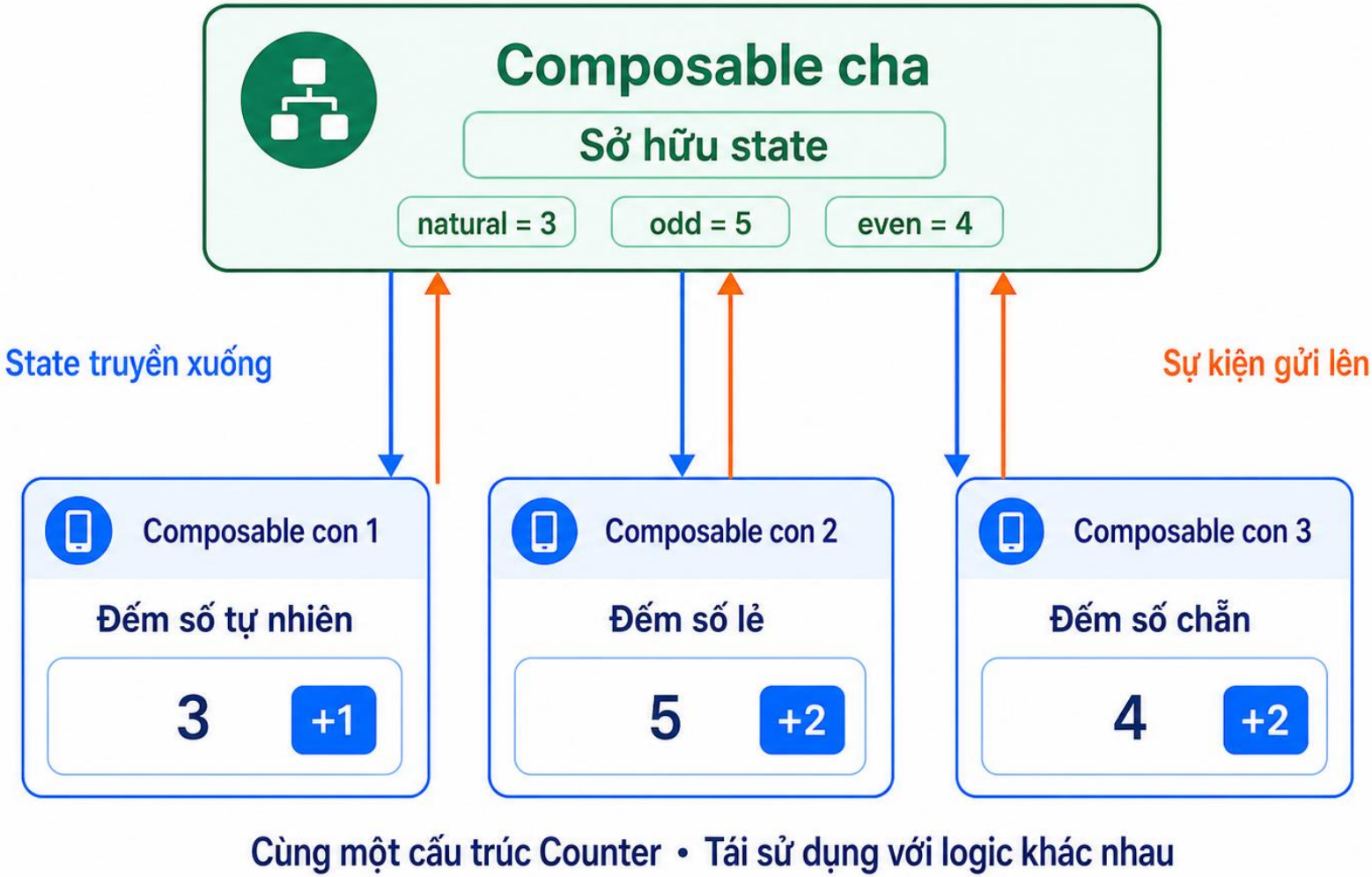
- Nhận value
- Phát callback onValueChange
- Dễ tái sử dụng, preview và test

Derived states

```
var amountInput by remember { mutableStateOf("") }
val amount = amountInput.toDoubleOrNull() ?: 0.0
val tip = calculateTip(amount)
```

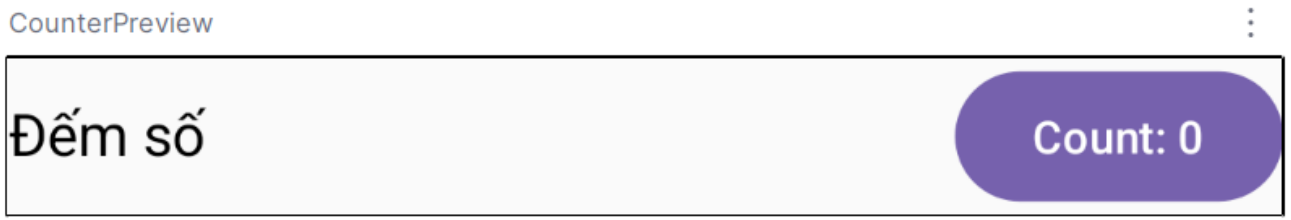
Cho phép suy diễn từ các state hiện có.

Tăng tính tổng quát và tái sử dụng



Ví dụ

```
@Composable
fun Counter(title: String, count: Int, onIncrement: ()->Unit) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Text(
            text = title,
            fontSize = 18.sp,
            modifier = Modifier.weight(1f))
        Button(onClick = onIncrement) {
            Text("Count: $count")
        }
    }
}
```



Ví dụ

```
@Composable
fun CounterApp() {
    var counter1 by remember { mutableIntStateOf(1) }
    var counter2 by remember { mutableIntStateOf(1) }
    var counter3 by remember { mutableIntStateOf(2) }
    Column {
        Counter("Đếm số tự nhiên", counter1) {}
        Counter("Đếm số lẻ", counter2) {}
        Counter("Đếm số chẵn", counter3) {}
    }
}
```



Resource annotations

```
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    ...
)
```

Resource annotation cung cấp thông tin kiểu bổ sung cho resource ID, giúp Android Studio/Lint phát hiện việc truyền sai loại resource.

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-calculate-tip

AUTOMATED TESTS

Xác nhận logic và hành vi UI lặp lại được.

Cấu trúc AAA: Arrange–Act–Assert

Arrange

Khởi tạo dữ liệu kiểm thử

Act

Thực hiện hàm, hành vi cần kiểm thử

Assert

Kiểm tra kết quả

Mỗi test nên tập trung vào một hành vi quan sát được.

Local test và Instrumented test

Tiêu chí	Local test	Instrumented test
Thư mục	src/test/	src/androidTest/
Nơi chạy	JVM trên máy tính	Thiết bị thật hoặc emulator
Tốc độ	Nhanh	Chậm hơn
Android Framework	Không có đầy đủ	Có
Phù hợp	Logic, hàm tính toán, ViewModel	Giao diện Compose, Activity, Navigation, Room

Unit test cho calculateTip

Mã cần kiểm thử

```
fun calculateTip(amount: Double, tipPercent: Double): Double {
    return amount * tipPercent / 100
}
```

Unit test cho calculateTip

Mã kiểm thử

```
@Test
fun calculateTip_20Percent_returns20() {
    // Arrange – Chuẩn bị dữ liệu kiểm thử
    val amount = 100.0
    val tipPercent = 20.0

    // Act – Gọi hàm cần kiểm tra
    val actual = calculateTip(amount = amount, tipPercent =
tipPercent)

    // Assert – So sánh kết quả thực tế với kết quả mong đợi
    assertEquals(20.0, actual, 0.001)
}
```

Compose UI test

Composable cần kiểm thử

```
@Composable
fun Counter(count: Int, onIncrement: () -> Unit) {
    Column {
        Text("Count: $count")
        Button(onClick = onIncrement) {
            Text("Increase")
        }
    }
}
```

Compose UI test

Kiểm thử tương tác

```
class CounterTest {
    @get:Rule
    val composeTestRule = createComposeRule()
    @Test
    fun clickIncrease_updatesCounter() {
        composeTestRule.setContent {
            var count by remember { mutableIntStateOf(0) }
            Counter(count = count,  onIncrement = { count++ })
        }
        composeTestRule.onNodeWithText("Count: 0").assertIsDisplayed()
        composeTestRule.onNodeWithText("Increase").performClick()
        composeTestRule.onNodeWithText("Count: 1").assertIsDisplayed()
    }
}
```

Codelab

Tạo ứng dụng Art Space

https://developer.android.com/codelabs/basic-android-kotlin-compose-art-space



HUST

hust.edu.vn fb.com/dhbkhn

Q&A